Session 2: Advanced Palawan Land Cover Lab

Multi-temporal Classification and Change Detection

Dimitrios Kasampalis
Anefelo

Session Overview

Duration: 2 hours
Type: Advanced Hands-on Lab
Focus: Real-world NRM Application

Study Area:
Central Palawan, 1° × 1° box (~12,000 km²)

Prerequisites: Session 1 completed, GEE authenticated

What You'll Learn:

- Advanced feature engineering (GLCM texture)
- Multi-temporal composites (dry/wet season)
- Hyperparameter optimization
- Deforestation detection (2020-2024)
- Protected area monitoring

Part A: Advanced Feature Engineering

Beyond Spectral Indices

Session 1 Features: - Spectral bands (B2-B12) - Vegetation indices (NDVI, EVI) - Water indices (NDWI, MNDWI) - Built-up index (NDBI)

Session 2 Advanced Features: - **Texture** (GLCM) - **Temporal** (seasonal composites) - **Topographic** (DEM-derived)

Goal: Measure how much each feature group adds

GLCM Texture Features

What is GLCM?

Gray-Level Co-occurrence Matrix measures spatial relationships between pixel pairs.

Why Use It?

- Distinguishes **primary vs secondary forest** (canopy structure differences)
- Separates **urban from bare soil** (heterogeneity patterns)
- Identifies **mangrove stands** (unique texture signature)
- Adds context that spectral values alone miss

GLCM Texture Metrics

Contrast - Measures local variation - High for heterogeneous areas (urban, mixed forest) - Low for uniform areas (water, dense forest)

Entropy - Measures randomness - High for complex textures - Low for regular patterns

Correlation - Measures pixel relationships - Detects linear structures - Useful for roads, rivers

Homogeneity - Measures uniformity - High for smooth surfaces - Low for rough textures

GEE GLCM Implementation

```
# glcmTexture needs integers: quantise NIR reflectance (0–0.5) to 0–255
image_nir = composite.select('B8').multiply(255 / 0.5).clamp(0, 255).toUint8()

# Calculate GLCM (size=1 → 3x3 window)
texture = image_nir.glcmTexture(size=1)

# Select key metrics
contrast = texture.select('B8_contrast')
entropy = texture.select('B8_ent')
correlation = texture.select('B8_corr')
homogeneity = texture.select('B8_idm')

# Add to feature stack
features = features.addBands([contrast, entropy,
                              correlation, homogeneity])
```

Computational Note: GLCM is intensive - use 3×3 windows for large areas

Multi-temporal Composites

Philippine Seasons

Dry Season (Dec-May)

- Less cloud cover (best for mapping)
- Maximum agricultural activity
- Forest at baseline state
- Ideal for structural analysis

Best for: - Forest type classification - Infrastructure detection - Land cover baseline

Wet Season (Jun-Nov)

- More cloud challenges
- Maximum vegetation vigor
- Rice fields flooded/growing
- Seasonal wetlands visible

Best for: - Agricultural identification - Crop phenology - Irrigated vs rainfed - Wetland mapping

Temporal Indices

NDVI Difference (Wet - Dry):

```
# Create seasonal composites
dry_composite = s2.filterDate('2024-01-01', '2024-05-31').median()
wet_composite = s2.filterDate('2024-06-01', '2024-11-30').median()

# Calculate NDVI for each
dry_ndvi = dry_composite.normalizedDifference(['B8', 'B4'])
wet_ndvi = wet_composite.normalizedDifference(['B8', 'B4'])

# Temporal difference
ndvi_diff = wet_ndvi.subtract(dry_ndvi)
```

Interpretation: - **Positive (>0.2):** Seasonal crops (rice) 🌾 - **Near zero (-0.1 to 0.1):** Evergreen forest 🌳 - **Negative (<-0.1):** Dry season crops, deciduous

Topographic Features

Why Add Elevation Data?

- **Altitude patterns:** Upland forest vs lowland agriculture
- **Slope:** Flat = agriculture, steep = forest (less accessible)
- **Aspect:** North-facing = more moisture = denser forest
- **Accessibility:** Low elevation near roads = higher deforestation risk

GEE Implementation:

```
# Load SRTM DEM
dem = ee.Image('USGS/SRTMGL1_003')

# Calculate derivatives
elevation = dem.select('elevation')
slope = ee.Terrain.slope(dem)
aspect = ee.Terrain.aspect(dem)

# Add to features
features = features.addBands([elevation, slope, aspect])
```

Palawan 8-Class Scheme

Classification Classes

Class	Description	Key Discriminators
🌳 **Primary Forest**	Dense dipterocarp, closed canopy	High NDVI, low texture, high elevation
🌲 **Secondary Forest**	Regenerating, mixed canopy	Moderate NDVI, medium texture
🌊 **Mangroves**	Coastal, tidal	High NDVI + high NDWI + coastal
🌾 **Agricultural**	Rice, coconut	Seasonal NDVI change, flat terrain
🌿 **Grassland**	Open, sparse vegetation	Low-moderate NDVI, low texture
💧 **Water**	Rivers, lakes, coastal	Very low NIR, high NDWI
🏘️ **Urban**	Settlements, infrastructure	High NDBI, high texture, low NDVI
⛏️ **Bare Soil**	Mining, cleared	Bright, low NDVI, near roads

Feature Stack Summary

Complete Feature Set (~21 features):

Spectral (6) - B2 (Blue) - B3 (Green) - B4 (Red) - B8 (NIR) - B11 (SWIR1) - B12 (SWIR2)

Indices (4) - NDVI - NDWI - NDBI - EVI

Texture (4) - Contrast - Entropy - Correlation - Homogeneity

Temporal (4) - Dry NDVI - Wet NDVI - NDVI difference - Seasonal amplitude

Topographic (3) - Elevation - Slope - Aspect

Hyperparameter Optimization

Random Forest Parameters

Key Parameters to Tune:

Parameter	Default	Range to Test	Impact
`numberOfTrees`	100	50, 100, 200, 500	More = better (diminishing returns)
`variablesPerSplit`	$\sqrt{n}$	$\sqrt{n}$, $\log_2(n)$, $n/3$	Balance randomness vs accuracy
`minLeafPopulation`	1	1, 2, 5, 10	Higher = simpler trees
`bagFraction`	0.5	0.5, 0.7, 1.0	Sampling strategy

Cross-Validation Strategy

K-Fold Cross-Validation (k=5):

```
# Assign each sample to one of 5 folds (randomColumn gives 0–1 floats)
folds = training_data.randomColumn('rand', seed=42).map(
    lambda f: f.set('fold', ee.Number(f.get('rand')).multiply(5).floor()))

# Test each fold
accuracies = []
for i in range(5):
    train = folds.filter(ee.Filter.neq('fold', i))
    test = folds.filter(ee.Filter.eq('fold', i))

    # Train model
    classifier = ee.Classifier.smileRandomForest(100).train(train, 'class', bands)

    # Evaluate
    accuracy = test.classify(classifier).errorMatrix('class', 'classification').accuracy()
    accuracies.append(accuracy.getInfo())

# Average accuracy
```

Handling Class Imbalance

The Problem:

Real-world datasets often have imbalanced classes (illustrative numbers):

Class	Pixels	Percentage
Primary Forest	450,000	45%
Secondary Forest	200,000	20%
Agriculture	180,000	18%
Water	100,000	10%
Grassland	40,000	4%
Urban	**15,000**	**1.5%** ← Rare class
Mangroves	**10,000**	**1%** ← Very rare
Bare Soil	5,000	0.5%

Consequence: Model ignores rare classes, poor accuracy for minority classes

Class Imbalance Solutions

1. Balanced Sampling

Oversample rare classes, undersample common classes:

```
# Equal samples per class
samples_per_class = 100

balanced_training = ee.FeatureCollection([])
for class_id in [1, 2, 3, 4, 5, 6, 7, 8]:
    class_samples = training.filter(
        ee.Filter.eq('class', class_id)
    ).randomColumn('random').limit(samples_per_class)
    balanced_training = balanced_training.merge(class_samples)
```

Pros: Simple, effective **Cons:** May oversample noisy pixels

2. Class Weights

Give higher weight to rare classes during training:

```
# Calculate class weights inversely proportional to frequency
class_counts = {1: 450000, 2: 200000, ..., 7: 10000, 8: 5000}
total = sum(class_counts.values())
weights = {k: total/(len(class_counts)*v) for k,v in class_counts.ite

# Not directly supported in GEE, but can weight samples
```

3. Stratified Validation

Ensure all classes in train AND test sets:

```
# Stratified split per class
training = ee.FeatureCollection([])
testing = ee.FeatureCollection([])

for class_id in [1, 2, 3, 4, 5, 6, 7, 8]:
    class_data = all_data.filter(ee.Filter.eq('class', class_id))
    class_data = class_data.randomColumn('random')

    train = class_data.filter(ee.Filter.lt('random', 0.8))
    test = class_data.filter(ee.Filter.gte('random', 0.8))

    training = training.merge(train)
    testing = testing.merge(test)
```

Out-of-Bag (OOB) Error

Built-in Validation:

Random Forest automatically provides OOB error estimate

```
# Train as usual
classifier = ee.Classifier.smileRandomForest(numberOfTrees=100) \
    .train(training, 'class', bands)

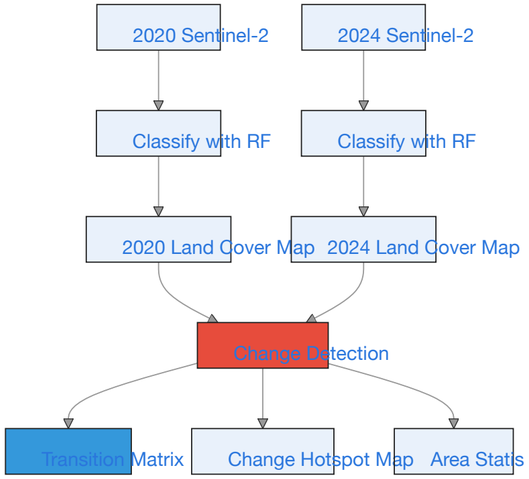
# OOB error is reported by explain()
# (confusionMatrix() is the *training/resubstitution* matrix, not OOB)
oob_error = classifier.explain().get('outOfBagErrorEstimate')
print('OOB error:', oob_error.getInfo())
```

Advantage: No need for separate validation set (~37% of data used for OOB)

Change Detection

2020 vs 2024 Comparison

Deforestation Analysis Workflow:



Change Detection Implementation

```
# Classify both years with same model
lc_2020 = composite_2020.classify(trained_classifier)
lc_2024 = composite_2024.classify(trained_classifier)

# Detect changes
change = lc_2024.subtract(lc_2020)

# Forest loss (class 1 or 2 → any other class)
forest_2020 = lc_2020.lte(2)  # Primary or secondary
forest_2024 = lc_2024.lte(2)
forest_loss = forest_2020.And(forest_2024.Not())

# Agricultural expansion
ag_gain = lc_2020.neq(4).And(lc_2024.eq(4))

# Calculate areas
forest_loss_area = forest_loss.multiply(ee.Image.pixelArea()).reduceRegion({
    reducer: ee.Reducer.sum(),
```

Transition Matrix

From-To Analysis (illustrative):

	→ Forest	→ Ag	→ Urban	→ Bare
Forest ↓	**85%**	12%	2%	1%
Ag ↓	3%	**90%**	5%	2%
Grassland ↓	8%	25%	**60%**	7%
Bare ↓	2%	10%	5%	**83%**

Key Insights: - 12% forest → agriculture (main driver) - 5% agriculture → urban (development) - Grassland mostly stable or converts to agriculture

Post-Processing Techniques

Why Post-Process Classifications?

Raw Classification Issues:

Common Problems: - **Salt-and-pepper noise:** Isolated misclassified pixels - **Small patches:** Below minimum mapping unit - **Edge effects:** Mixed pixels at boundaries - **Geometric errors:** Irregular shapes

Solutions: 1. Majority/Modal filter 2. Minimum mapping unit filter 3. Morphological operations 4. Boundary smoothing

Goal: Clean, cartographically appealing maps suitable for stakeholder communication

Majority Filter (Focal Mode)

Smooth noisy pixels using neighborhood majority:

```
# Apply 3x3 majority filter
classification = classification_raw.focal_mode(
    radius=1,  # 3x3 window (1 pixel in each direction)
    kernelType='square'
)

# More aggressive: 5x5 filter
classification_smooth = classification_raw.focal_mode(
    radius=2,  # 5x5 window
    kernelType='square'
)
```

Before Filtering: - Salt-and-pepper noise - Isolated pixels - Fragmented patches

After Filtering: - Smoother appearance - More contiguous patches - Reduced noise

Trade-off: May lose small but real features (small clearings, narrow roads)

Minimum Mapping Unit (MMU)

Remove patches smaller than threshold:

```
# Step 1: Connected component labeling
connected = classification.connectedPixelCount(maxSize=256)

# Step 2: Filter by size (e.g., MMU = 25 pixels = 0.25 ha at 10m resolution)
mmu_threshold = 25
classification_mmu = classification.updateMask(connected.gte(mmu_threshold))

# Step 3: Fill gaps with focal_mode
classification_clean = classification_mmu.focal_mode(radius=5, kernelType='square')
```

MMU Guidelines:

Local (1:10,000)	0.1 - 0.5 ha	Detailed management
Regional (1:50,000)	1 - 5 ha	Provincial planning
National (1:250,000)	10 - 25 ha	National reporting

Morphological Operations

Opening and Closing for shape refinement:

Opening (Erosion → Dilation):

Removes small protrusions, separates narrow connections

```
# Erode then dilate
eroded = classification.focal_min(radius=1)
opened = eroded.focal_max(radius=1)
```

Use for: - Breaking thin connections - Removing speckles - Smoothing boundaries

Closing (Dilation → Erosion):

Fills small holes, connects nearby patches

```
# Dilate then erode
dilated = classification.focal_max(radius=1)
closed = dilated.focal_min(radius=1)
```

Use for: - Filling gaps - Joining nearby patches - Closing boundaries

Combined Workflow:

```
# Remove noise (opening) then fill gaps (closing)
classification_clean = classification.focal_min(1).focal_max(1)  # Opening
classification_final = classification_clean.focal_max(1).focal_min(1)  # Closing
```

Expert Rules & Constraints

Apply domain knowledge to refine results:

```
# Rule 1: Mangroves only near coast
mangrove_class = 3
distance_to_coast = coastline.distance(maxDistance=5000)  # 5km
mangrove_mask = classification.eq(mangrove_class).And(distance_to_coast.lt(2000))
classification = classification.where(
    classification.eq(mangrove_class).And(distance_to_coast.gte(2000)),
    2  # Reclassify as secondary forest
)

# Rule 2: Agriculture unlikely above 1000m elevation
ag_class = 4
classification = classification.where(
    classification.eq(ag_class).And(elevation.gt(1000)),
    5  # Reclassify as grassland
)

# Rule 3: Urban near roads
urban_class = 6
```

Post-Processing Workflow Summary

Recommended Pipeline:



Quality Control Checklist: - ✓ Visual inspection of before/after - ✓ Area statistics comparison (should be similar) - ✓ Check for over-smoothing - ✓ Validate against high-resolution imagery - ✓ Ensure ecologically plausible results

Protected Area Monitoring

Palawan Conservation Context

UNESCO Biosphere Reserve (1990)

Biodiversity: - Often called the Philippines' "last ecological frontier" - Many endemic species (e.g. Palawan peacock-pheasant) - Critical habitat for endangered species

Area: - Core: 3,000 km² - Buffer: 5,000 km² - Transition: 3,655 km²

Threats: - Mining (nickel, chromite) - Agricultural expansion - Infrastructure (roads, ports) - Illegal logging - Tourism pressure

Management: - DENR oversight - Strategic Environmental Plan (SEP) - Local government units (LGUs) - NGO partnerships

Encroachment Detection

Boundary Analysis:

```python
# Load protected area boundary
protected_area = ee.FeatureCollection('path/to/palawan_PA')

# Create buffer zones
core = protected_area
buffer_1km = core.buffer(1000)
buffer_5km = core.buffer(5000)

# Detect forest loss in each zone
core_loss = forest_loss.clip(core)
buffer_loss = forest_loss.clip(buffer_5km).subtract(core_loss)

# Calculate statistics
core_loss_area = core_loss.multiply(ee.Image.pixelArea()).reduceRegion(...)
buffer_loss_area = buffer_loss.multiply(ee.Image.pixelArea()).reduceRegion(...)

# Generate alert if threshold exceeded
if core_loss_area > threshold:
```

Deforestation Hotspot Map

Kernel Density Analysis:

```
# Identify forest loss pixels
loss_pixels = forest_loss.selfMask()

# Convert to points
loss_points = loss_pixels.sample(
    region=aoi,
    scale=10,
    geometries=True
)

# Kernel density estimation
hotspots = loss_points.reduceToImage(['classification'],
                                     ee.Reducer.count())
                      .convolve(ee.Kernel.gaussian(500))

# Visualize
Map.addLayer(hotspots, {min: 0, max: 50, palette: ['white', 'yellow', 'red']},
             'Deforestation Hotspots')
```

Use Case: Target field verification and enforcement

Expected Outcomes

Performance Targets

Accuracy Improvement (illustrative — depends on labels and area):

Approach	Overall Accuracy	Kappa
Session 1 (Basic RF)	82%	0.78
+ GLCM Texture	85%	0.82
+ Multi-temporal	87%	0.84
+ Topographic	**88-90%**	**0.86-0.88**

Per-Class Targets: >85% for most classes

Common Confusions

Expected Confusion Pairs:

1. **Primary ↔ Secondary Forest**
   - Similar spectral signature
   - Texture helps but overlap exists
   - Solution: Add canopy height (LiDAR)
2. **Mangrove ↔ Wet Season Agriculture**
   - Both high NDVI + water proximity
   - Solution: Temporal analysis (mangroves stable)
3. **Urban ↔ Bare Soil**
   - Both bright in visible bands
   - Solution: Texture (urban more heterogeneous)

Session Deliverables

By the end of this session:

✅ High-resolution land cover map (10m Palawan)
✅ Comprehensive accuracy report (per-class metrics)
✅ Feature importance analysis
✅ 2020-2024 change detection map
✅ Deforestation statistics (hectares per class)
✅ Hotspot map for DENR
✅ Exported GeoTIFF for QGIS integration
✅ Area statistics CSV

NRM Applications

DENR Use Cases

Forest Monitoring: - Annual forest cover updates - REDD+ MRV compliance - Protected area assessment - Illegal logging detection

Implementation: - Automated monthly processing - Alert system for >5 ha forest loss - Integration with field teams - Reporting to central office

Local Government Applications

Land Use Planning: - Zoning map updates - Infrastructure siting (avoid sensitive areas) - Agricultural zone delineation - Tourism planning

Disaster Risk: - Flood-prone areas (based on land cover) - Landslide susceptibility (slope + forest loss) - Evacuation route planning

NGO Conservation Programs

Community Monitoring: - Train local rangers to use maps - Mobile app for ground truthing - Participatory mapping sessions - Livelihood integration (agroforestry zones)

Impact Assessment: - Baseline for conservation projects - Monitor restoration success - Detect encroachment early - Evidence for advocacy

Technical Considerations

Computational Challenges

GEE Limitations:

⚠ Common Errors

“Computation timed out” - **Cause:** GLCM on large area - **Solution:** Process in tiles, export intermediates

“Memory limit exceeded” - **Cause:** Too many features + large AOI - **Solution:** Reduce feature count, use `.aside()` sparingly

“User memory limit exceeded” - **Cause:** Complex reducers - **Solution:** Simplify, use `.limit()` on collections

Optimization Strategies

Speed Up Processing:

1. **Use .limit()** on ImageCollections
2. **Export intermediate results** (composites, features)
3. **Reduce GLCM window** (5×5 → 3×3)
4. **Process by tiles** for very large areas
5. **Use .aside()** judiciously for debugging

Example:

```
# Instead of:
composite = collection.median()  # Slow

# Do:
composite = collection.limit(50).median()  # Faster, usually sufficient
```

Summary

Key Takeaways

1. **Feature Engineering Matters:** texture, temporal and terrain features typically add several points
2. **Multi-temporal is Powerful:** Seasonal patterns reveal agriculture vs forest
3. **GLCM Adds Context:** Spatial structure complements spectral info
4. **Hyperparameter Tuning:** Small gains but worth it for production
5. **Change Detection:** Quantifies deforestation for stakeholders
6. **Real-world Relevance:** the same building blocks underpin operational forest and land cover monitoring

Next Steps

After Session 2

Immediate: - Complete all notebook exercises - Experiment with feature combinations - Try different AOIs in Philippines

Session 3 Preview: Deep Learning and CNNs - automatic feature learning!

Continue to Session 3 →

Questions & Resources

Documentation: - GEE GLCM - RF Classifier - Hansen Global Forest Change

Support: - Instructor Q&A - GEE Forum - Session notebook with complete code